

Round Robin (RR)

Round Robin gives each ready process a turn on the CPU. An unfinished process goes to the back of the queue when its turn ends.

1. Service time and time slice

The visualizer uses one CPU, whole ticks, and CPU-only processes. Switching processes takes no time.

Service time is the total CPU work a process needs to finish. The **quantum**, also called the **time slice**, limits how many CPU ticks it can use in one turn. The default quantum is 2; any positive whole number is allowed.

For example, a process needing five service ticks with a quantum of 2 needs turns of two, two, and one tick. Waiting between turns does not reduce its remaining service. RR has no separate allotment or priority levels.

2. Scheduling rules

Rule 1. Add arrivals to the queue tail

Existing waiting processes stay ahead of new arrivals. If several processes arrive together, add them in input order. If arrival and quantum expiry coincide, admit the arrivals before requeueing the unfinished process.

Rule 2. Give the queue head a fresh quantum

When the CPU is free, remove the first ready process and set its quantum usage to zero. Each tick it runs reduces remaining service by one and increases quantum usage by one.

An arriving process cannot interrupt a turn. It waits until the current process finishes or uses its quantum.

Rule 3. Complete before considering rotation

If the process has no service left, remove it. This applies even when its last service tick also uses the last tick of its quantum. A finished process is never sent back to the queue.

Rule 4. Rotate an unfinished process when its quantum expires

Reset its quantum usage and append it to the queue tail. Preserve its remaining service. If other processes are waiting, they run first.

If the process is alone, it can immediately start another turn. Quantum expiry does not create an idle tick.

Rule 5. Idle only when no process is ready

Wait for the next arrival if the CPU and ready queue are empty. Stop when all processes have finished.

3. Events at a tick boundary

After accounting for the preceding tick:

1. Remove the running process if it has finished.
2. If an unfinished process has used its quantum, mark its turn as ended.
3. Add all arrivals at the current time.
4. Append the process whose turn ended, if any, with quantum usage reset.
5. If the CPU is free, dispatch the queue head.

If the running process has neither finished nor exhausted its quantum, it continues. Dispatching or requeueing takes no CPU tick.

For example, suppose A's turn ends while B is waiting and C arrives. Admit C, then append A. The queue becomes [B, C, A], with B next.

4. Worked examples

Example 1. Full turns and early completion

Input order is A, B, C. Use quantum 2. All times are in ticks.

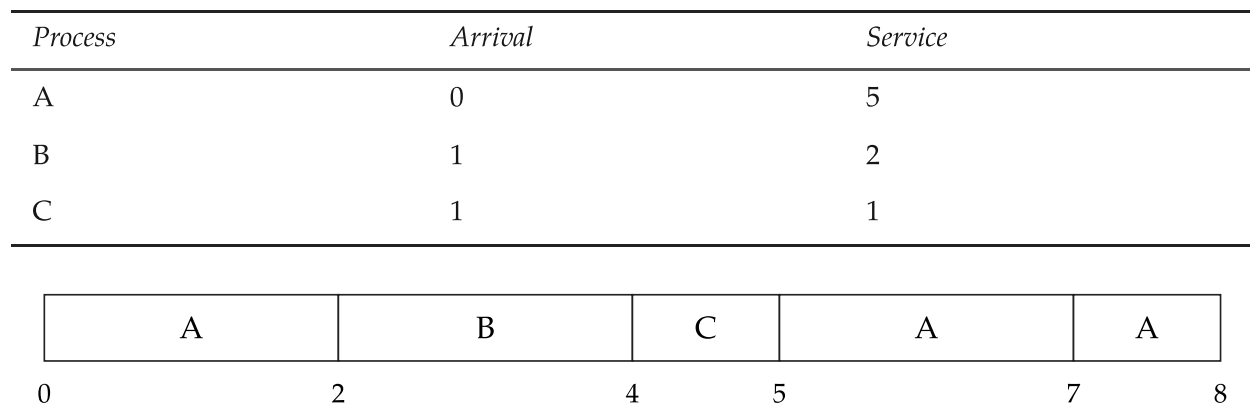


Figure 1. CPU schedule (time in ticks).

A's first turn ends at time 2, with three service ticks left. B then finishes exactly at its quantum boundary. C needs only one tick, so it releases the CPU at time 5 without using its full quantum. A runs from 5 to 7, then immediately takes another turn because nobody else is waiting. It finishes at time 8.

The boundary at time 7 separates two turns of A, although A uses the CPU on both sides.

Example 2. Arrival at quantum expiry

Let A arrive at time 0 needing four ticks, and B arrive at time 2 needing one. Keep quantum 2.

The CPU runs A from time 0 to 2, B from 2 to 3, then A from 3 to 5.

At time 2, B is admitted before A is requeued. The queue is [B, A], so B runs next. A resumes at time 3 with two service ticks left.